A **SUM** statement takes the form:

SELECT SUM (Field) – this specifies the field (column) for the calculation. The field should be integer or real.

A **COUNT** statement takes the form:

SELECT COUNT (Field) – this specifies the field (column) to count if the given criterium is met.

Example 1: Display consultant's patients

For example, the following SQL command for the PATIENT single-table database would provide a list of all Mr Smith's patients showing the hospital number, first name and family name for each of his patients.

```
SELECT HospitalNumber, FirstName, FamilyName
FROM PATIENT
WHERE Consultant = 'Mr Smith';
```

would display:

HospitalNumber	FirstName	FamilyName
HN123456	Winnie	Sing
HN123458	Min	Hoo

▲ **Figure 9.5** Output from the PATIENT table showing Mr Smith's patients

Example 2: Display consultant's patients in alphabetical order

This SQL command sorts the records in alphabetical order of family name:

```
SELECT HospitalNumber, FirstName, FamilyName
FROM PATIENT
WHERE Consultant = 'Mr Smith'
ORDER BY FamilyName;
```

would display:

HospitalNumber	FirstName	FamilyName
HN123458	Min	Hoo
HN123456	Winnie	Sing

▲ **Figure 9.6** Output from the PATIENT table showing Mr Smith's patients in alphabetical order of family name

Activity 9.5

1 Using the single table database PATIENT you have created.
   a Write an SQL query to list all Mr Jones' patients.
   b Write an SQL query to list all the patients not in ward 6.
   c Write an SQL query to list all the patients who arrived on 12/11/2022.
   d Write an SQL query to list all the patients who arrived between 12/10/2022 **AND** 30/10/2022.
2 Write down the output from this SQL query.

```
SELECT FirstName, FamilyName, BedNumber
FROM PATIENT
WHERE WardNumber = 7;
```

Activity 9.6

1 Using the single table database PATIENT you have created.
   a Write an SQL query to count the number of patients in ward 7.
   b Write an SQL query to count the number of patients not in ward 7.
2 Write down the output from this SQL query.

```
SELECT HospitalNumber, FirstName, FamilyName, Consultant
FROM PATIENT
ORDER BY Consultant, FamilyName;
```

Practical use of a database

As an IGCSE Computer Science student you need to be able to do the following:

- define a single-table database from given data storage requirements
- choose a suitable primary key for a database table
- read, complete and understand SQL scripts.

In order to do this, you will need to use a database management system. The following case study shows how to set up a database with *Microsoft Access 365* and complete the tasks described above.

Boys and girls between the ages of seven and eleven can join a cub scout group. (http://en.wikipedia.org/wiki/Cub_Scout). Each cub scout group needs to keep records about its members. Most groups will keep the following information about each cub in their group:

Activity 9.7

1 **a** Set up a cub scout database including appropriate validation checks for each field.

Enter data for at least 10 records.

b The cub scout leader wants to put each cub into a group called a 'six'; each 'six' can have up to six cubs in it and is given a name for example red, yellow, blue and green.

Add a new text field called Six, and assign each cub to either a red, blue, yellow or green six.

i Write an SQL query to pick out any cubs in the red six.

ii Write an SQL query to pick out any cubs in the red six or the blue six.

iii Write an SQL query to count the number of cubs in the red six.

c The cub scout leader wants to calculate the number of badges that all the cubs have been awarded. Add a new integer field called *Badges*, enter the number of badges awarded to each cub.

Write an SQL query to count the number of badges awarded to the whole cub group.

Activity 9.8

1 A database of students is to be set up with the following fields:

- Family name
- Other names
- Student ID
- Date of Birth
- Date of Entry to School
- Current Class
- Current school year/grade
- Email address.

a Select a data type for each field.

b Which fields should be validated, and which fields should be verified?

c Decide the validation rules for those fields which should be validated.

d Which field would you choose for the primary key?

e Choose a suitable format for the student ID.

f Build a database with at least 10 records; include all your validation checks. Ensure there are at least 3 different classes and 2 different years/grades.

g Set up and test SQL scripts to:

i Display Other names, Family and Email address in alphabetical order of family name.

ii Select all the students from each class in alphabetical order.

iii Select all the students for each year/grade and print Other names, Family name and Date of Birth, grouping the students by class.

Link

For more on verification, see Chapter 7.